

PL/SQL Practical Assignment Programs

Assignment - 5 | Complete Clean Code Implementation

1. Program to Print "Welcome To PL/SQL Block"

```
SET SERVEROUTPUT ON;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Welcome To PL/SQL Block');
END;
/
```

2. Program to Count Square

```
SET SERVEROUTPUT ON;
DECLARE
    v_num NUMBER := 6;
    v_sq  NUMBER;
BEGIN
    v_sq := v_num * v_num;
    DBMS_OUTPUT.PUT_LINE('Square of ' || v_num || ' is: ' || v_sq);
END;
/
```

3. Program to Find Simple Interest

```
SET SERVEROUTPUT ON;
DECLARE
    v_p  NUMBER := 10000;
    v_r  NUMBER := 5;
    v_t  NUMBER := 2;
    v_si NUMBER;
BEGIN
    v_si := (v_p * v_r * v_t) / 100;
    DBMS_OUTPUT.PUT_LINE('Simple Interest is: ' || v_si);
END;
/
```

4. Program for Global and Local Variable

```
SET SERVEROUTPUT ON;
DECLARE
    v_global NUMBER := 100;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Outer Variable: ' || v_global);
    DECLARE
        v_local NUMBER := 50;
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Inner Local Variable: ' || v_local);
        DBMS_OUTPUT.PUT_LINE('Accessing Global inside inner block: ' || v_global);
    END;
END;
/
```

5. Program to Declare a Constant Variable

```
SET SERVEROUTPUT ON;
DECLARE
    v_pi CONSTANT NUMBER(3,2) := 3.14;
    v_r  NUMBER := 5;
    v_area NUMBER;
BEGIN
    v_area := v_pi * v_r * v_r;
    DBMS_OUTPUT.PUT_LINE('Area of Circle: ' || v_area);
END;
/
```

6. Check Ticket Age-wise using IF-THEN-ELSIF-ELSE

```
SET SERVEROUTPUT ON;
DECLARE
    v_age NUMBER := 15;
BEGIN
    IF v_age < 5 THEN
        DBMS_OUTPUT.PUT_LINE('Ticket is Free');
    ELSIF v_age BETWEEN 5 AND 12 THEN
        DBMS_OUTPUT.PUT_LINE('Child Ticket Price: $5');
    ELSIF v_age BETWEEN 13 AND 59 THEN
        DBMS_OUTPUT.PUT_LINE('Adult Ticket Price: $12');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Senior Ticket Price: $8');
    END IF;
END;
/
```

7. Input Three Numbers and Find Maximum and Minimum

```
SET SERVEROUTPUT ON;
DECLARE
    a NUMBER := 45;
    b NUMBER := 78;
    c NUMBER := 23;
    v_max NUMBER;
    v_min NUMBER;
BEGIN
    v_max := GREATEST(a, b, c);
    v_min := LEAST(a, b, c);
    DBMS_OUTPUT.PUT_LINE('Maximum Number is: ' || v_max);
    DBMS_OUTPUT.PUT_LINE('Minimum Number is: ' || v_min);
END;
/
```

8. Total, Percentage, Result, and Grade for a Student

```
SET SERVEROUTPUT ON;
DECLARE
    v_rollno NUMBER := 101;
    m1 NUMBER := 85;
    m2 NUMBER := 90;
    m3 NUMBER := 88;
    v_total NUMBER;
    v_per NUMBER;
    v_grade VARCHAR2(5);
BEGIN
    v_total := m1 + m2 + m3;
    v_per := v_total / 3;
    IF v_per >= 75 THEN v_grade := 'A';
    ELSIF v_per >= 60 THEN v_grade := 'B';
    ELSIF v_per >= 50 THEN v_grade := 'C';
    ELSE v_grade := 'D';
    END IF;
    DBMS_OUTPUT.PUT_LINE('Roll No: ' || v_rollno);
    DBMS_OUTPUT.PUT_LINE('Total Marks: ' || v_total);
    DBMS_OUTPUT.PUT_LINE('Percentage: ' || v_per || '%');
    DBMS_OUTPUT.PUT_LINE('Grade: ' || v_grade);
END;
/
```

9. Program for PL/SQL Case Statement

```
SET SERVEROUTPUT ON;
DECLARE
    v_grade CHAR(1) := 'B';
BEGIN
    CASE v_grade
        WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Excellent');
        WHEN 'B' THEN DBMS_OUTPUT.PUT_LINE('Very Good');
        WHEN 'C' THEN DBMS_OUTPUT.PUT_LINE('Good');
        ELSE DBMS_OUTPUT.PUT_LINE('Needs Improvement');
    END CASE;
END;
/
```

10. Print First 10 Numbers Using Basic Loop..Exit Loop

```
SET SERVEROUTPUT ON;
DECLARE
    i NUMBER := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        EXIT WHEN i = 10;
        i := i + 1;
    END LOOP;
END;
/
```

11. Print First 10 Numbers Using While Loop

```
SET SERVEROUTPUT ON;
DECLARE
    i NUMBER := 1;
BEGIN
    WHILE i <= 10 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        i := i + 1;
    END LOOP;
END;
/
```

12. Print First 10 Numbers Using For Loop

```
SET SERVEROUTPUT ON;
BEGIN
    FOR i IN 1..10 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;
END;
/
```

13. Print First 1 to 10 Numbers in Reverse Using For Loop

```
SET SERVEROUTPUT ON;
BEGIN
    FOR i IN REVERSE 1..10 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;
END;
/
```

14. Print First 10 Prime Numbers Using While Loop

```
SET SERVEROUTPUT ON;
DECLARE
    n NUMBER := 2;
    count_prime NUMBER := 0;
    is_prime BOOLEAN;
    j NUMBER;
BEGIN
    WHILE count_prime < 10 LOOP
        is_prime := TRUE;
        j := 2;
        WHILE j <= TRUNC(SQRT(n)) LOOP
            IF MOD(n, j) = 0 THEN is_prime := FALSE; EXIT; END IF;
            j := j + 1;
        END LOOP;
        IF is_prime THEN
            DBMS_OUTPUT.PUT_LINE(n);
            count_prime := count_prime + 1;
        END IF;
        n := n + 1;
    END LOOP;
END;
/
```

15. Accept a String and Display Reverse of It

```
SET SERVEROUTPUT ON;
DECLARE
    v_str VARCHAR2(50) := 'WELCOME';
    v_rev VARCHAR2(50);
BEGIN
    FOR i IN REVERSE 1..LENGTH(v_str) LOOP
        v_rev := v_rev || SUBSTR(v_str, i, 1);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Original: ' || v_str);
    DBMS_OUTPUT.PUT_LINE('Reversed: ' || v_rev);
END;
/
```

16. Input a String and Display Its Pyramid

```
SET SERVEROUTPUT ON;
DECLARE
    v_str VARCHAR2(20) := 'SQL';
    v_len NUMBER;
BEGIN
    v_len := LENGTH(v_str);
    FOR i IN 1..v_len LOOP
        DBMS_OUTPUT.PUT_LINE(LPAD(SUBSTR(v_str, 1, i), v_len + i - 1, ' '));
    END LOOP;
END;
/
```

17. Print Number Pyramid

```
SET SERVEROUTPUT ON;
BEGIN
  FOR i IN 1..4 LOOP
    DBMS_OUTPUT.PUT_LINE(LPAD(' ', 5 - i) ||
      CASE i
        WHEN 1 THEN '1'
        WHEN 2 THEN '1 2'
        WHEN 3 THEN '1 2 3'
        WHEN 4 THEN '1 2 3 4'
      END);
  END LOOP;
END;
/
```

18. Program for the Use of GOTO Statement

```
SET SERVEROUTPUT ON;
DECLARE
  v_val NUMBER := 1;
BEGIN
  <<start_loop>>
  DBMS_OUTPUT.PUT_LINE('Value: ' || v_val);
  v_val := v_val + 1;
  IF v_val <= 3 THEN
    GOTO start_loop;
  END IF;
  DBMS_OUTPUT.PUT_LINE('End of GOTO loop');
END;
/
```

19. Insert 10 Rows into Square Table

```
-- Prerequisites: CREATE TABLE square_table (num NUMBER, sq NUMBER);
SET SERVEROUTPUT ON;
BEGIN
  FOR i IN 1..10 LOOP
    INSERT INTO square_table VALUES (i, i * i);
  END LOOP;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('10 rows inserted successfully.');
```

```
END;
/
```

20. Update Row of Square Table

```
SET SERVEROUTPUT ON;
BEGIN
  FOR i IN 1..10 LOOP
    UPDATE square_table SET sq = i * i * 10 WHERE num = i;
  END LOOP;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Rows updated successfully.');
```

END;
/

21. Delete Row from Square Table

```
SET SERVEROUTPUT ON;
BEGIN
  FOR i IN 1..5 LOOP
    DELETE FROM square_table WHERE num = i;
  END LOOP;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Rows deleted successfully.');
```

END;
/